

Instituto Tecnológico de Estudios Superiores de
Occidente



ITESO, Universidad Jesuita de Guadalajara

Internet de las Cosas (IoT)
Profesor: Ricardo Zamora Dávalos

A2. Avance semana 12

Ana Isabel Guillen González
Diego Fernando Pérez González

Noviembre, 2025

Contenido

Definición	3
Problemática	3
Objetivos (general y específico).....	3
Alcance	4
Justificación	4
Investigación	5
Metodología y Alcance	6
Propuesta del diseño del Sistema (diagrama)	6
Herramientas	7
Hardware.....	7
Software.....	7
Planeación	8
Recursos	8
Riesgos	9
Entregables	10
Documentación	11
Avances semana 8.....	12
Pruebas de funcionamiento del hardware	12
Avance planificado	14
Problemáticas encontradas y su solución.....	16
Avances semana 12	20
Avance planificado	20
Problemáticas encontradas y su solución.....	24
Estado actual del proyecto	26
Bibliografía:	27

Definición

Como propuesta de proyecto en primera instancia, lo que se busca desarrollar es un sistema que funja como asistente de voz (Voice Assistan). De principio dicho proyecto se llamaría **(Local) Home Security Voice Assistant**. Dicho proyecto busca enfocarse en el control de luces, la detección de un timbre y activación de una cámara, todo esto acompañado de una GUI sencilla que permita mostrar al usuario lo que está activo y/o funcionando, así como las imágenes captadas con la cámara.

Problemática

La problemática de la que nace la idea del proyecto es la inseguridad que necesita ser atendida en los hogares, específicamente hablando de la entrada. Por otro lado, y quitando a un lado el factor de seguridad, está también el tema de practicidad a la hora de saber quién está timbrando o queriendo entrar a la casa sin que necesariamente uno esté cerca de la puerta tenga que asomarse al picaporte o de plano abrir para saber quién es.

Objetivos (general y específico)

En general, buscamos desarrollar un asistente de voz local para el hogar que integre funciones de control de iluminación, detección de timbre, activación de cámara y visualización en una interfaz gráfica, con el fin de aumentar la seguridad y la practicidad en el acceso a la entrada de la vivienda mediante un sistema económico y privado.

En aspectos más específicos y concretos, tenemos los siguientes objetivos:

- **Diseñar e implementar un módulo de control de luces por comandos de voz**, que permita al usuario gestionar la iluminación de la entrada del hogar.
- **Integrar un sistema de detección de timbre** que active automáticamente la cámara y capture imágenes o video del visitante.
- **Desarrollar una interfaz gráfica sencilla e intuitiva** que muestre al usuario el estado de los dispositivos (luces, timbre, cámara) y las imágenes captadas.
- **Optimizar el sistema para que sea de bajo costo y accesible**, priorizando el uso de hardware embebido eficiente y asequible.

Alcance

El presente proyecto tendrá como alcance:

1. Plataforma de desarrollo

- Implementación del sistema sobre la tarjeta embebida **PICO-PI-IMX8M-MINI** utilizando como sistema operativo **Ubuntu Linux**.
- Integración de librerías y frameworks compatibles con Linux para el manejo de audio, cámara y GUI.

2. Funcionalidades principales

- **Control de luces por voz:** comandos básicos de encendido y apagado.
- **Detección de timbre:** activación automática de la cámara al presionar el timbre.
- **Captura y visualización de imágenes** de la entrada mediante la cámara conectada al sistema.
- **Interfaz gráfica (GUI)** sencilla para mostrar el estado de luces, timbre y cámara, así como las imágenes captadas.
- **Procesamiento local de comandos de voz**, sin conexión a servicios en la nube, priorizando la seguridad y privacidad del usuario.

3. Restricciones y exclusiones

- El proyecto no busca competir con asistentes comerciales como Alexa o Google Home, por lo que no incluirá integración con servicios externos en la nube.
- No se implementará control general de dispositivos domóticos de toda la casa, únicamente el entorno de la entrada principal (luces, timbre y cámara).
- El sistema se enfocará en una versión prototipo funcional y no en un producto comercial final.

Justificación

Una de las justificaciones con las que contamos para proceder con el desarrollo de este proyecto es poder ofrecer un sistema asistido por voz de este tipo (sistema “portero” o de timbre-cámara-pantalla) de bajo costo en la medida de lo posible. También se buscar ofrecer un sistema local que por ende sea y se perciba por el usuario más seguro, esto en contraste con asistentes de voz de grandes corporativos como Alexa, en dónde se procesa la voz en servidores remotos, implicando entonces, que audios y posibles conversaciones sensibles son

enviadas a la nube. Con un servidor local nos aseguramos de procesar todo con nuestro hardware [4].

Investigación

La aplicación en aspectos generales claramente ya existe, sin considerar los requerimientos y usos más específicos que le buscamos dar al proyecto, aquí hay algunos ejemplos y referencias generales:

- **“Build a Voice Assistant with TensorFlow Lite”**, asistente de voz que funcione sin conexión a internet, ejecutando el reconocimiento de comandos directamente en un microcontrolador [1]
- **“OpenVoiceOS (OVOS)”** es una plataforma comunitaria, de código abierto y enfocada en la privacidad, diseñada para crear interfaces de voz personalizables en diversos dispositivos. Permite control por voz, integración con dispositivos embebidos [2].
- **“i.MX Voice Player”**. Proyecto que muestra cómo realizar control por voz mediante Bluetooth en placas de la familia i.MX8 con soporte para QT GUI, BlueZ y PulseAudio. Aunque no se centra en la placa utilizada en la clase, podría tener código de referencia interesante [3].
- **“Asistente de voz Alexa”** Es un servicio de voz que se puede encontrar en una gran variedad de dispositivos, principalmente en los dispositivos de Amazon, con este asistente es posible dar una gran cantidad de comandos al decir su “Wake Word” seguido por el comando, en el cual se pueden realizar preguntas para que el asistente conteste o peticiones de control del hogar, en dispositivos como luces, termostatos, etc. [5]

Tomando la investigación previa como base, se puede confirmar que el desarrollo de este proyecto es posible, mediante la utilización de la tarjeta i.MX8 mini pico, ya que en esta tarjeta se pueden utilizar recursos que accedan a una red local de internet para utilizar el sistema de reconocimiento de voz, mostrar una GUI en su display (incluido en el kit de desarrollo), y utilizar un hardware externo simple que nos permita visualizar el efecto de los comandos de voz en leds, un push-button para el timbre y la activación de una cámara de seguridad.

Además, la i.MX8MM puede correr Ubuntu 20.04 o una distro basada en Yocto, lo que permite instalar librerías necesarias para el desarrollo del proyecto, el SDK maneja captura de voz, tiene capacidad de audio y manejo de GPIOs.

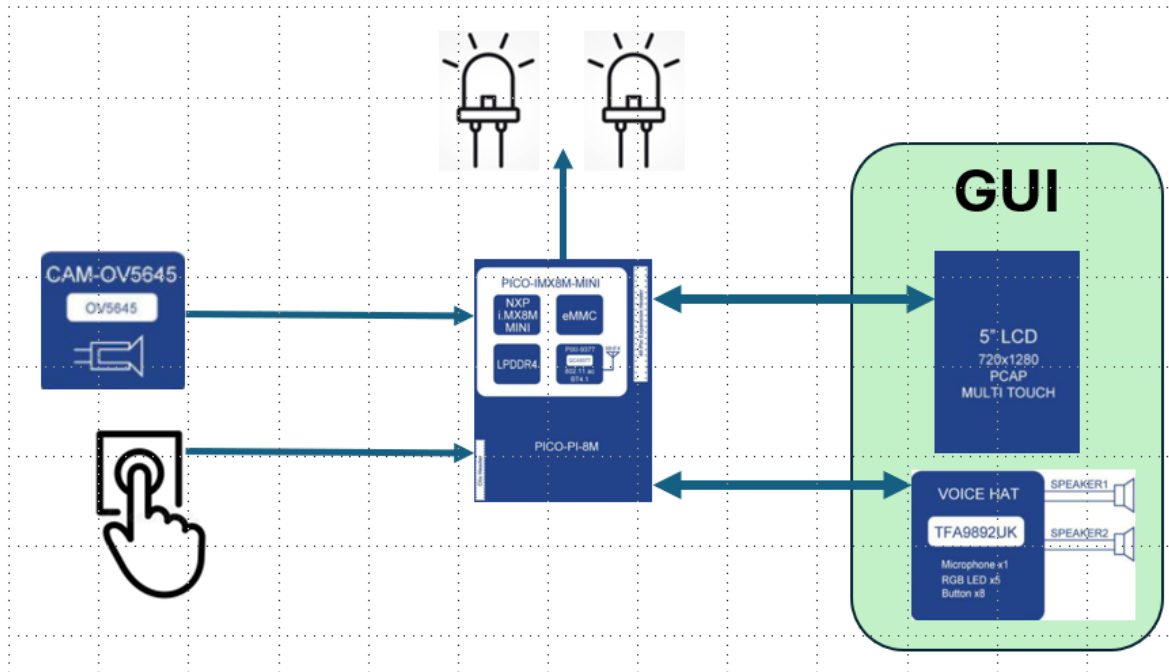
Con esto buscamos enfocarnos principalmente en el reconocimiento y procesamiento de voz, demostrando sus resultados mediante el uso de leds.

Metodología y Alcance

El proyecto a desarrollar cuenta con diferentes etapas a completar para que la implementación conjunta del proyecto funcione de manera adecuada, lo cual se puede dividir de la siguiente manera:

- Funcionamiento de Hardware externo, implica la detección adecuada de un led externo y un push-button.
- Funcionamiento del Hardware incluido en el kit de desarrollo, el display y la cámara.
- Uso del software de reconocimiento de voz.
- Desarrollo de la GUI.
- Integración del led con el reconocimiento de voz.
- Integración del push-button con la cámara.
- Integración de los elementos del sistema.

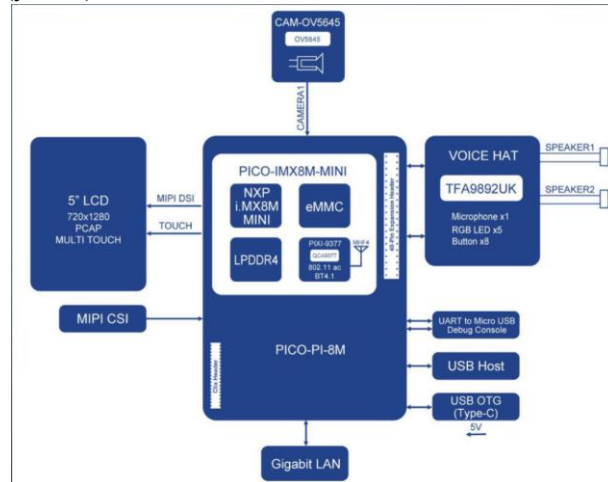
Propuesta del diseño del Sistema (diagrama)



Herramientas

Hardware

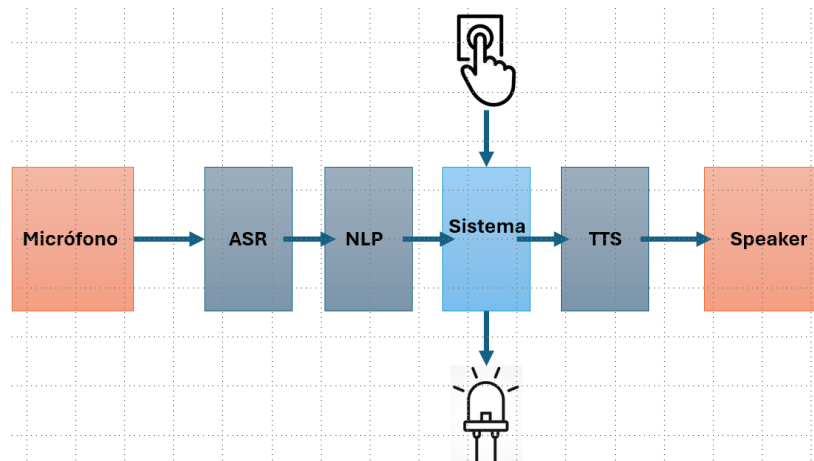
- Placa de desarrollo (y KIT) PICO-PI-IMX8M-MINI:



- Cables de alimentación para la board.
- Display (incluido en el kit de desarrollo).
- Cámara (incluida en el set de desarrollo).
- Push-button.
- Led.

Software

- Sistema Operativo **Ubuntu Linux**
- Middleware** (todavía por elegir/encontrar).
- ASR** (Automatic Speech Recognition), módulo de SW encargado de convertir Voz en Texto. Todavía por escoger...
- NLP** (Procesador de lenguaje natural o Reconocimiento de Intención). Aún por escoger...
- TTS**, convertidor de texto generado por el sistema a voz.



Como primera estancia se estima utilizar el servicio de Alexa de AWS para la implementación del reconocimiento de voz.

- Plataforma de desarrollo de la GUI.
- GitHub.
- Generación de código en lenguajes de programación.

Planeación

A continuación se presenta un planificador de fechas en el cual se establecerá el desarrollo del proyecto.

#	Actividad Central	Mes 1 (semanas)				Mes 2 (semanas)				Mes 3 (semanas)				Mes 4 (semanas)			
		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
1	Definición del proyecto y documentación inicial	x	x	x	x												
2	Definición de plataformas a utilizar					x											
3	Desarrollo del funcionamiento de Hardware y base del uso de de SW para el reconocimiento de voz						x	x	x								
4	Desarrollo de la GUI e Integración del Push-button con la cámara								x	x	x						
5	Integración del LED con el reconocimiento de voz											x	x				
6	Integración de todos los elementos del sistema															x	
7	Documentación final y presentación del proyecto																x

Semana 1 a 4: Definición del proyecto, documentación inicial y pruebas del funcionamiento del Hardware incluido en el kit de desarrollo, el display y la cámara.

Semana 5: Definición de plataformas a utilizar para la elaboración de la GUI y el reconocimiento de voz.

Semana 6 a 8: Desarrollo del funcionamiento de Hardware externo, la detección adecuada de un led externo y un push-button, bases para el uso del software de reconocimiento de voz.

Semana 8 a 10: Desarrollo de la GUI e integración del push-button con la cámara.

Semana 11 a 12: Integración del led con el reconocimiento de voz.

Semana 15: Integración de los elementos del sistema.

Semana 16: Documentación final y presentación del proyecto.

Recursos

Para cumplir con los elementos mencionados en la planeación es necesario contar con diversos recursos, donde además del factor humano, formado por el equipo de desarrollo se

requieren de elementos de Hardware y Software, los cuales se mencionaron en secciones previas, por lo cual, en esta sección se hará un recuento de si se cuenta o no con el recurso.

- Hardware

Recurso	Disponibilidad
Tarjeta i.MX8MM pico	Si
Cables de alimentación	Si
Display	Si
Cámara	Si
Leds	Si
Push-Buttons	Si
Resistencias	Si
Bocina	Si

- Software

Recurso	Disponibilidad
Ubuntu 20.4	Si
Prueba de funcionamiento de display	Si
Prueba de funcionamiento de cámara	Si
Plataforma de reconocimiento de voz	Servicio de Alexa de AWS
Plataforma de desarrollo de la GUI	No definido aún
Código de control de leds	No
Código de detección de push-button	

Riesgos

- Problemas en la integración del Hardware externo como el push-button y los leds a la tarjeta.
- Fallas en la detección de la red WiFi en la tarjeta.
- Plataforma de reconocimiento de voz inestable (que no escuche Wake Word y comandos o que escuche basura y genere comandos inexistentes).
- Creación de GUI en plataforma no compatible con el sistema a desarrollar,
- Problemas de integración del sistema en conjunto.

Entregables

Semana	Entregable	Criterios de evaluación	Criterios de éxito
4	* Documento con la planeación del proyecto	Cumple con los apartados de: - Definición. - Investigación. - Metodología y Alcance. - Planeación. - Entregables. - Documentación.	Todos los apartados se encuentran dentro del documento. Cada apartado se encuentra completo. Cada apartado contiene información relevante sobre el proyecto. Se cubre toda la información solicitada en cada apartado.
8	* Detección adecuada de un led externo * Detección adecuada de un push-button, * Primeras pruebas del uso del software de reconocimiento de voz	* Se cuenta con un código bien elaborado el cual detecte una señal específica y con eso se accione un led. * Se cuenta con un código bien elaborado el cual detecte cuando se presiona el push-button. * El sistema detecta la Wakeword a utilizar de manera adecuada.	* El led responde con el estado correspondiente a la simulación del comando, es decir, que encienda y apague correctamente. * Se detecta el botón en tiempo real y se responde con una señal específica cuando este se acciona. * Se ve una respuesta en la tarjeta cada que se menciona la Wakeword del sistema.
12	* Integración del push-button con la cámara. * Integración del led con el reconocimiento de voz.	* Se cuenta con un código que enlace el uso de la cámara al push-button. * Se cuenta con un código que vincule el encendido y apagado del led al reconocimiento de voz.	* Al presionar el botón se activa la cámara por un tiempo determinado. * Si el usuario le menciona al sistema que encienda el led, este debe de prender, y si se menciona que lo apague, el led se debe apagar.
15	* Integración de los elementos del sistema. * Diseño de la GUI.	* Se cuenta con un código unificado en el cual se realice la acción de la cámara y reconocimiento de voz. * Se tiene el diseño de la GUI.	* Se realizan las acciones del código de manera adecuada. Se encienden y apagan los leds según el comando de voz y la cámara se activa solo con el push-button. * La GUI se encuentra realizada.
16	* Proyecto terminado.	* Todo el Proyecto esta unificado. * La GUI refleja las acciones que se ejecutan. * La documentación esta completa.	* El sistema funciona en tiempo real. * Los elementos no interfieren entre sí. * La documentación detalla cómo utilizar el proyecto.

Documentación

La documentación del proyecto consistirá en diversos elementos, en los cuales se describa cómo es que se ha avanzado en este proyecto a lo largo del tiempo, y lo que se planea hacer para continuar en su desarrollo.

La documentación se llevará a cabo mediante este documento, en el cual se registrará la parte teórica del proyecto, es decir, el registro de avances, dentro de lo cual se encuentra:

- Definición y especificaciones de la plataforma de reconocimiento de voz a utilizar.
- Definición y especificaciones de la plataforma en la cual se realizará la GUI.
- Definición y especificaciones del Hardware a utilizar.
- Definición y especificaciones del Software.
- Fotografías de los avances realizados.
- Listado de las acciones realizadas.
- Problemas encontrados.
- Cambios en la planificación del proyecto.

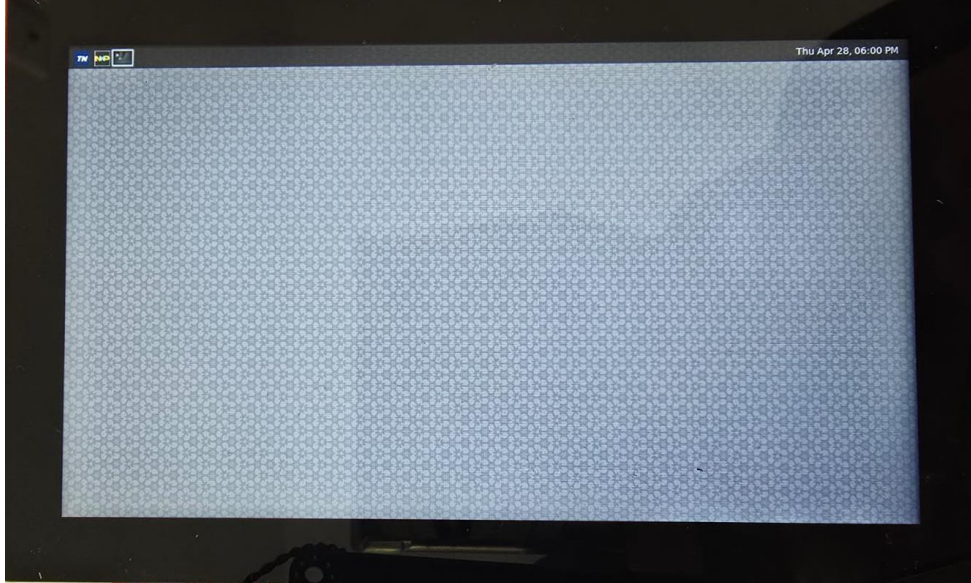
Mientras que en Github (https://github.com/FerRafiki/IoT_Project_Equipo5) se realizará el control de versiones del proyecto, dentro de lo cual se encuentra:

- Código.
- Makefiles.
- Configuraciones de las plataformas utilizadas.
- Información acerca de las nuevas implementaciones de cada avance.
- Logs con información relevante en el desarrollo del proyecto.

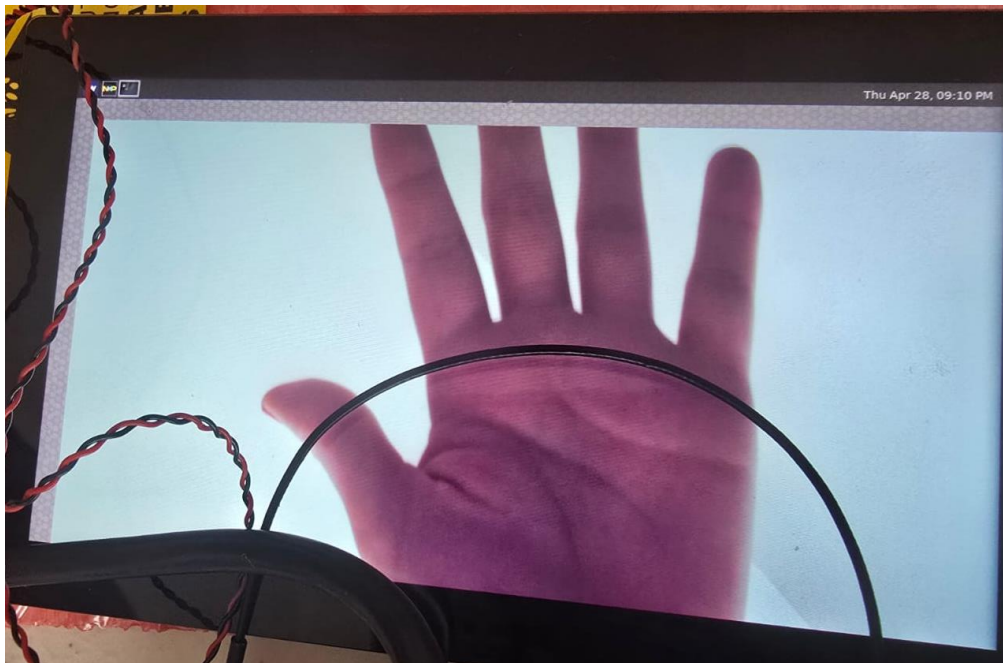
Avances semana 8

Pruebas de funcionamiento del hardware

- Display



- Cámara



- Voice Hat

Grabación

```
root@pico-imx8mm:~# ./audio_capture timbre.wav hw:0
[ 43.943293] imx_sph0645_startup():
[ 43.947734] imx_sph0645_trigger():
^C[ 50.316277] imx_sph0645_trigger():
[ 50.319885] imx_sph0645_shutdown():
```

```

^A^K^I^A^E^C^|^AG^I^A^E^|^A^E?^|A^E||^A^E4^|A^E|^|^A^E|^|^A^E$|^A^|^A^E|^|^A
e|^C^e^U
e^e^W^D^e
e^e^U^D^e
e^d^e^e^j
e6^D^eQ
e^e<^D^ei
eN^D^e^e
eU^D^e^e
em^D^e
ez^D^e^e
e^e>^D^e
e^D^e^e^u
e^e^D^e^|
e^e||^D^e^
e||^D^e^e=
e^eJ^D^e^
e^u^D^e^e^
e^e█^D^e^e
a^k^e^e|^D^e^x^k^e|^D^e^eH^k^e^e^D^e^eI^k^e^e^^E^eB^k^e^J^E^ey^k^e*^E^e^e
G^e^M^e^e^e^U^G^e^u^M^e1^G^e^M^e*^G^e^e^M^e^e;^G^e^e^M^e^eQ^G^e^M^ed^G
e|^P^e^K
e^e^J^Q^e^U
e^e8^Q^e^U
e^e?^Q^e^e$
e^e/^Q^e8
e^EN^Q^e0
eF^Q^eC
c^EN^Q^e^eI
ey^Q^eI
c^e^eg^Q^e^e1
e^Q^e
e^Q^e^e
f^e%Q^e^e!
- ana.wav 1/2011 0%
```

Reproducción

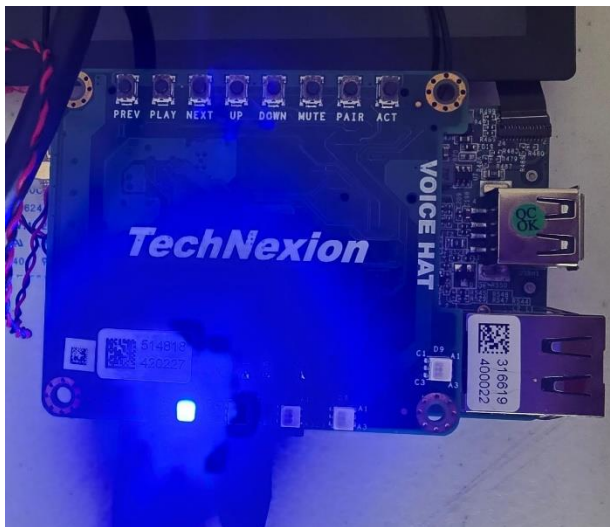
```
root@pico-imx8mm:~# ./playback timbre.wav plughw:1
[ 70.963455]
[ 70.967358]
[ 77.319651]
[ 77.325530]
```

Conclusión:

Al realizar estas pruebas fue posible confirmar que el hardware externo a utilizar funciona de manera adecuada, por lo tanto, es posible continuar con la ejecución del proyecto.

Avance planificado

Detección adecuada de un led externo



Para hacer funcionar el led fue necesario entrar al archivo .dts del voicehat para comprobar que el funcionamiento de sus leds se compone de instrucciones en i2c.

Para prender el led se realizó el script `led_on.sh` y para apagar el led es `led_off.sh`, mediante el uso de estos scripts es posible prender y apagar el dependiendo el uso que se le quiera dar.

Detección adecuada de un push-button

```
Escuchando botones:
PREV presionado
[ 56.675085 ]
[ 56.683146 ]
[ 58.080771 ]
[ 58.086688 ]
PREV liberado
ACT presionado
ACT presionado
PAIR presionado
MUTE presionado
MUTE liberado
VOL- presionado
VOL+ presionado
NEXT presionado
PLAY presionado
PLAY liberado
PREV presionado
[ 70.136482 ]
[ 70.140079 ]
[ 71.537697 ]
[ 71.543515 ]
PREV liberado
```

Basándonos en el comando `evtest /dev/input/event0` se pudo encontrar el valor de cada botón del voicehat y registrarlo mediante un código, `buttons.c` en el cual se registra la detección de botones, con el cual se vio que cada uno de estos funciona de manera adecuada.

Se utilizó una variable fd (file descriptor) de solo lectura el cual pudiera detectar los cambios en el evento seleccionado y a partir de eso generar un print al detectar un botón.

Este código es al momento, la base del proyecto, ya que dependiendo el botón presionado se realiza una acción.

Primeras pruebas del uso del software de reconocimiento de voz

Empezamos haciendo pruebas para que por lo menos la “wake word” funcionara, una vez logrando esto, modificando el código y haciendo pruebas para ver qué palabras dentro del vocabulario con el que está entrenado el reconocedor de voz eran más factibles o fáciles de ser detectadas, llegamos a hasta el momento a este conjunto mínimo de palabras ya probadas para nuestro sistema:

- “entrance” (para despertar el reconocimiento al resto de las palabras clave)
- “on” (para prender nuestro led)
- “down” (para apagar nuestro led)

Aquí sabemos que un mejor uso de vocabulario hubiera sido “off”, sin embargo, haciendo pruebas nos dimos cuenta de que esta palabra era de difícil detección.

- “exit” (para desactivar el reconocimiento de voz)

```
if recognizer.AcceptWaveform(data):
    result = json.loads(recognizer.Result())
    text = result.get("text", "").lower()
    #print(f"Recognized: {text}")

    if not text:
        continue

    if not is_active:
        if "entrance" in text:
            is_active = True
            print("Activated! Say 'on' to turn ON the LED or 'down' to turn OFF the LED")
        else:
            if "on" in text:
                print("LED ON")
                #is_active = False
                clear_audio_queue()
                reset_recognizer()
            elif "down" in text:
                print("LED OFF")
                #is_active = False
                clear_audio_queue()
                reset_recognizer()
            elif "exit" in text:
                print("Exiting.")
                break

except Exception as e:
    print(f"Error in audio stream: {e}")
```

```
diegofer@ubuntu:~/vosk_models$ python3 voice_assistant_vosk.py
LOG (VoskAPI:ReadDataFiles():model.cc:213) Decoding params beam=10 max-active=3000 lattice-beam=2
LOG (VoskAPI:ReadDataFiles():model.cc:216) Silence phones 1:2:3:4:5:6:7:8:9:10
LOG (VoskAPI:RemoveOrphanNodes():nnet-nnet.cc:948) Removed 0 orphan nodes.
LOG (VoskAPI:RemoveOrphanComponents():nnet-nnet.cc:847) Removing 0 orphan components.
LOG (VoskAPI:ReadDataFiles():model.cc:248) Loading l-vector extractor from /home/diegofer/vosk_models/model_en/ivector/final.ie
LOG (VoskAPI:ComputeDerivedVars():ivector-extractor.cc:183) Computing derived variables for lVector extractor
LOG (VoskAPI:ComputeDerivedVars():ivector-extractor.cc:204) Done.
LOG (VoskAPI:ReadDataFiles():model.cc:282) Loading HCL and G from /home/diegofer/vosk_models/model_en/graph/HCLr.fst /home/diegofer/vosk_models/model_en/graph/Gr.fst
LOG (VoskAPI:ReadDataFiles():model.cc:308) Loading winfo /home/diegofer/vosk_models/model_en/graph/phones/word_boundary.lnt
Say 'entrance' to activate voice control...
Activated! Say 'on' to turn ON the LED or 'down' to turn OFF the LED
LED ON
LED OFF
Exiting.
diegofer@ubuntu:~/vosk_models$
```


Problemáticas encontradas y su solución

Funcionamiento del playback

Se encontró que al conectar y desconectar el voicehat, reiniciar la board o al desconectarla, muchas veces cambia el hardware al cual esta conectada la bocina, provocando que en gran cantidad de ocasiones se de un segmentation fault, se abra el speaker, entre otras acciones que impedían la reproducción del sonido del timbre.

Para solucionar esto se implemento el script doorbell.sh, en el cual se ejecuta la acción de detectar en qué hw se encuentra la bocina TFA y guardar esto en una variable card_num, de manera, en la que no importa si es hw:0, o hw:1, el hardware de la bocina se ajusta automáticamente, lo cual nos permite escuchar el sonido del timbre necesario en el proyecto.

Integración de Hardware

Se encontró que al conectar el voicehat y el display estos no eran compatibles, ya que si se quería ver el display, junto con la imagen de la cámara, los eventos relacionados al voicehat no funcionaban, por el contrario, si se hacía funcionar al voicehat borrando el elemento ili9881c, dejaba de funcionar el display. Por lo tanto se complicaba la integración necesaria del proyecto de display con voicehat.

Para solucionar esto se revisaron los dts relacionados al voicehat y al display, con lo que se encontró que existía un conflicto entre estos dispositivos debido al uso del gpio3:

Dts del Voicehat

```
gpio_keys {
    compatible = "gpio-keys";
    pinctrl-names = "default";
    pinctrl-0 = <&pinctrl_gpio_key_voicehat>;

    prev {
        gpios = <&gpio3 0 GPIO_ACTIVE_LOW>;
        label = "KEY PREV";
        linux,code = <KEY_PREVIOUS>;
    };

    play {
        gpios = <&gpio5 4 GPIO_ACTIVE_LOW>;
        label = "KEY PLAY";
        linux,code = <KEY_PLAY>;
    };

    next {
        gpios = <&gpio5 20 GPIO_ACTIVE_LOW>;
        label = "KEY NEXT";
        linux,code = <KEY_NEXT>;
    };

    up {
        gpios = <&gpio3 7 GPIO_ACTIVE_LOW>;
```

```
up {
    gpios = <&gpio3 7 GPIO_ACTIVE_LOW>;
    label = "KEY UP";
    linux,code = <KEY_UP>;
};

down {
    gpios = <&gpio3 16 GPIO_ACTIVE_LOW>;
    label = "KEY DOWN";
    linux,code = <KEY_DOWN>;
};

mute {
    gpios = <&gpio3 6 GPIO_ACTIVE_LOW>;
    label = "KEY MUTE";
    linux,code = <KEY_MUTE>;
};

pair {
    gpios = <&gpio5 13 GPIO_ACTIVE_LOW>;
    label = "KEY PAIR";
    linux,code = <KEY_SELECT>;
};

act {
    gpios = <&gpio5 3 GPIO_ACTIVE_LOW>;
    label = "KEY ACT";
    linux,code = <KEY_OK>;
};
```



```

&i2c3 {
    /* PCA9955 - 16-bit RGB LED */
    led_pca9955: pca9955@60 {
        #address-cells = <1>;
        #size-cells = <0>;
        compatible = "nxp,pca9955";
        reg = <0x60>;

        led0 {
            label = "blue0";
            linux,default-trigger = "none";
            reg = <0>;
        };

        led1 {
            label = "green0";
            linux,default-trigger = "none";
            reg = <1>;
        };

        led2 {
            label = "red0";
            linux,default-trigger = "none";
            reg = <2>;
        };

        led3 {

```

```

                                LPIX81
        led3 {
            label = "blue1";
            linux,default-trigger = "none";
            reg = <3>;
        };

        led4 {
            label = "green1";
            linux,default-trigger = "none";
            reg = <4>;
        };

        led5 {
            label = "red1";
            linux,default-trigger = "none";
            reg = <5>;
        };

        led6 {
            label = "blue2";
            linux,default-trigger = "none";
            reg = <6>;
        };

        led7 {
            label = "green2";
            linux,default-trigger = "none";
            reg = <7>;
        };

```

```

        led8 {
            label = "red2";
            linux,default-trigger = "none";
            reg = <8>;
        };

        led9 {
            label = "blue3";
            linux,default-trigger = "none";
            reg = <9>;
        };

        led10 {
            label = "green3";
            linux,default-trigger = "none";
            reg = <10>;
        };

        led11 {
            label = "red3";
            linux,default-trigger = "none";
            reg = <11>;
        };

        led12 {
            label = "blue4";
            linux,default-trigger = "none";
            reg = <12>;
        };
    };
}

```

Dts del display

```
&mipi_dsi {
    status = "okay";

    panel@0 {
        compatible = "bananapi,lhr050h41", "ilitek,ili9881c";
        pinctrl-0 = <&pinctrl_mipi_dsi_rst>;
        reset-gpio = <&gpio3 9 GPIO_ACTIVE_LOW>;
        reg = <0>;
        timing-mode = <0>;
        video-mode = <2>;          /* 0: burst mode
                                   * 1: non-burst mode with sync event
                                   * 2: non-burst mode with sync pulse
                                   */

        status = "okay";
    };
};
```

Con estas capturas es posible apreciar que varios botones usan el gpio3, al igual que el reset-gpio del display, por lo tanto la solución más simple que se encontró fue modificar el dts del display, cambiando el reset-gpio que utilizaba gpio3 por gpio1, quedando de la siguiente manera:

NUEVO ili9881c.dts

```
GNU nano 4.8                               imx8mm-pico-pi-ili9881c.dts
&mipi_dsi {
    status = "okay";

    panel@0 {
        compatible = "bananapi,lhr050h41", "ilitek,ili9881c";
        pinctrl-0 = <&pinctrl_mipi_dsi_rst>;
        reset-gpio = <&gpio1 9 GPIO_ACTIVE_LOW>;
        reg = <0>;
        timing-mode = <0>;
        video-mode = <2>;          /* 0: burst mode
                                   * 1: non-burst mode with sync event
                                   * 2: non-burst mode with sync pulse
                                   */

        status = "okay";
    };
};

&i2c3 {
    polytouch: edt-ft5x06@38 {
        compatible = "edt,edt-ft5x06";
        reg = <0x38>;
        pinctrl-names = "default";
    };
};
```

De manera posterior a este cambio se realizó la compilación de los dts para obtener un dtb final, el cual se mando a la tarjeta y se puede encontrar en la carpeta de dtb.

Con estos cambios se nos permitió realizar la integración del display con el voicehat de manera adecuada.

Reconocimiento de voz

Originalmente se planteaba utilizar el servicio de reconocimiento de voz AWS de Alexa, sin embargo, esta plataforma ya no permite la creación de cuentas de acceso libre, por lo cual, el plan original de reconocimiento de voz se vio afectado, retrasando el avance esperado en esta área del proyecto.

Para solucionar esto se investigaron diversas plataformas de reconocimiento de voz, entre ellas se encontró “Vosk”, plataforma que mediante la elaboración de un archivo de Python permite el reconocimiento de wakewords y comandos simples.

Actualmente, la etapa de reconocimiento de voz del proyecto se esta desarrollando en esta plataforma, lo cual se esta probando en la máquina virtual de Linux para asegurar su funcionamiento de manera previa a mandarlo a la board.

A rasgos generales, dentro de nuestro código en python hacemos uso de lo siguiente para poner en marcha nuestro reconocedor de voz:

- Capturador de audio “**sounddevice**”, Nos ayuda a capturar el audio del micrófono en tiempo real
- Librería “**queue**” para poder manejar una cola con los datos del audio
- Interpretador **json**, para interpretar los resultados del reconocimiento de voz
- Motor de reconocimiento de voz offline llamado **vosk**

```
1  import sounddevice as sd
2  import queue
3  import json
4  from vosk import Model, KaldiRecognizer
5
```

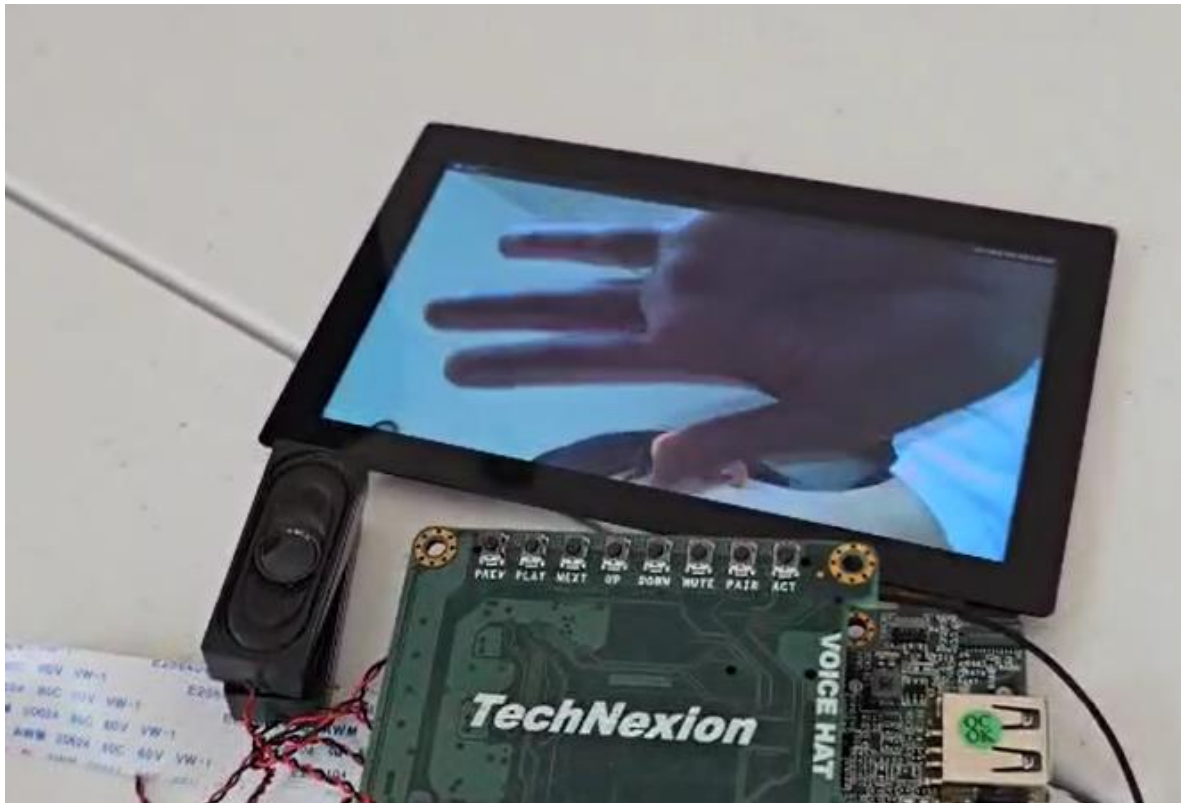
Avances semana 12

Avance planificado

Botón + cámara

Para este paso, se añadió el funcionamiento de la cámara en el código de detección de botones, el código buttons previamente mencionado, en este se definen los siguientes dos casos:

- PREV: Lanza el script camera.sh, el cual contiene el pipeline de gstreamer que le permite al i.MX8M Mini abrir y acceder a las imágenes de la cámara.
- PLAY: Cierra la cámara, matando el proceso que le permitía ejecutarse mediante pkill.



Este avance representa la acción de que al “timbrar” una persona en la casa, el sistema Home Security Voice Assistant, abre la cámara para que el usuario, pueda revisar quien lo está solicitando, y ya que el usuario logre ver quien es, el mismo pueda cerrar la cámara cuando considere conveniente.

Reconocimiento de voz en la board

El código para reconocimiento de voz en la board se elaboró de la siguiente manera, se tomo como base el código de python utilizado para probar los comandos de voz con el modelo VOSK.

Para adaptar el código se realizaron los siguientes cambios:

PC	BOARD
<code>import sounddevice as sd</code> Requerido para la captura de audio e la computadora.	<code>import subprocess</code> Librería adecuada para capturar audio en el i.MX8M mini.
<code>samplerate = 16000</code> En la computadora solo se requiere la frecuencia de muestreo del modelo de VOSK.	<code>rate_hw = 32000 # Frecuencia de SPH0645</code> <code>rate_vosk = 16000 # Frecuencia de Vosk</code> Sin embargo para nuestra aplicación en la board, el micrófono captura en formato LE32, por lo que se necesita definir la frecuencia del micrófono y del modelo VOSK.
<code>recognizer = KaldiRecognizer(model, samplerate)</code> En la PC solo es necesario activar el modelo de reconocimiento de voz.	<code>recognizer = KaldiRecognizer(model, rate_vosk, ["entrance", "on", "down", "exit"])</code> Debido a la sensibilidad de reconocimiento de voz en la board fue necesario limitar manualmente la cantidad de palabras que puede reconocer.
<code>audio_q = queue.Queue()</code> Modo de captura de audio.	<code>proc = subprocess.Popen(cmd, stdout=subprocess.PIPE, bufsize=0)</code> Modo de captura de audio.
Sin necesidad de proceso para normalizar el audio.	Necesita proceso para normalizar el audio.
No se conectaba a ningún HW externo	Utiliza la librería os para conectarse a la board y poder utilizar su sistema operativo para prender y apagar los leds
Exit cerraba totalmente el asistente de voz.	Con Exit se dejan de escuchar los comandos on y down pero no se cierra el asistente de voz, solo espera el siguiente actívale.

En pocas palabras se adaptó el código que ya se tenía para que funcionara según las especificaciones de HW de la board utilizada.

Funcionamiento inicial del reconocimiento de voz, simplemente imprimiendo la acción que se quería ejecutar y cerrando totalmente el asistente al decir “Exit”.

```
Escuchando micrófono (SPH0645)... di 'entrance' para activar control de voz
Reconocido: entrance
Voice control activado. Di 'on' para encender el LED o 'down' para apagarlo.
Reconocido: on
LED ON
Reconocido: on
LED ON
Reconocido: on
LED ON
Reconocido: down
LED OFF
Reconocido: down
LED OFF
Reconocido: down
LED OFF
Reconocido: on
LED ON
Reconocido: down
LED OFF
Reconocido: exit
Saliendo del asistente de voz.
[ 2751.441427] inx_sph0645_trigger():
[ 2751.445133] inx_sph0645_shutdown():
root@pico-imx8mm:~#
```

Vista del log completo al iniciar el asistente de voz.

En esta versión los comandos on y down ya activan y desactivan el led seleccionado del voicehat, y al mencionar “Exit” no se cierra el asistente de voz, si no que espera de nuevo a la WakeWord “Entrance”.

```
root@pico-imx8mm:~# python3 Home.py
LOG (VoskAPI:ReadDataFiles():model.cc:213) Decoding params beam=10 max-active=3000 lattice-beam=2
LOG (VoskAPI:ReadDataFiles():model.cc:216) Silence phones 1:2:3:4:5:6:7:8:9:10
LOG (VoskAPI:RemoveOrphanNodes():nnnet-nnet.cc:298) Removed 0 orphan nodes.
LOG (VoskAPI:RemoveOrphanComponents():nnnet-nnet.cc:847) Removing 0 orphan components.
LOG (VoskAPI:ReadDataFiles():model.cc:248) Loading i-vector extractor from /home/root/vosk_models/model_en/ivector/final.ie
LOG (VoskAPI:ComputeDerivedVars():ivector-extractor.cc:183) Computing derived variables for iVector extractor
LOG (VoskAPI:ComputeDerivedVars():ivector-extractor.cc:204) Done.
LOG (VoskAPI:ReadDataFiles():model.cc:282) Loading HGL and G from /home/root/vosk_models/model_en/graph/HGL.fst /home/root/vosk_models/model_en/graph/Gr.fst
LOG (VoskAPI:ReadDataFiles():model.cc:308) Loading winfo /home/root/vosk_models/model_en/graph/phones/word_boundary.int
LOG (L 5612.344159) inx_sph0645_startup():
VoskAPI:UpdateGrammarFst():recogn1 5612.349707] inx_sph0645_trigger():
ispr.cc:282) ['entrance', 'on', 'down', 'exit']
LOG (VoskAPI:Estimate():language_model.cc:142) Estimating language model with ngram-order=2, discount=0.5
LOG (VoskAPI:OutputFst():language_model.cc:209) Created language model with 5 states and 8 arcs.
Entrance para asistente de voz
Reconocido: entrance
'on' para encender el LED o 'down' para apagarlo.
Reconocido: on
LED ON
Reconocido: on
LED ON
Reconocido: on
LED ON
Reconocido: down
LED OFF
Reconocido: down
LED OFF
Reconocido: down
LED OFF
Reconocido: exit
Saliendo del asistente de voz.
Reconocido: entrance
'on' para encender el LED o 'down' para apagarlo.
Reconocido: on
LED ON
Reconocido: on
LED ON
Reconocido: down
LED OFF
Reconocido: down
LED OFF
Reconocido: exit
Saliendo del asistente de voz.
```

Integración de Hardware

Para integrar el reconocimiento de voz junto con el código “buttons” que reconoce las acciones de los botones, tales como prender la cámara, sonar el timbre, apagar la cámara, prender y apagar el led mediante botones; se realizó un script simple en el cual en primera instancia se activa el código de reconocimiento de voz y se corre de fonde utilizando “&” y posteriormente se activa el código buttons.

Con esto se tiene una base del proyecto funcional en la cual los leds se activan o desactivan tanto por comandos de voz o mediante el uso de botones.

Asimismo, el funcionamiento de la cámara ya se encuentra integrado mediante el uso de botones.

```
Eschando botones: PREU, PLAY, NEXT, VOL+, VOL-, MUTE, PAIR...
LOG (VoskAPI:ReadDataFiles():model.cc:213) Decoding params beam=10 max-active=3000 lattice-beam=2
LOG (VoskAPI:ReadDataFiles():model.cc:216) Silence phones 1:2:3:4:5:6:7:8:9:10
LOG (VoskAPI:RemoveOrphanNodes():nnet-mnet.cc:948) Removed 0 orphan nodes.
LOG (VoskAPI:RemoveOrphanComponents():nnet-mnet.cc:847) Removing 0 orphan components.
LOG (VoskAPI:ReadDataFiles():model.cc:248) Loading i-vector extractor from /home/root/vosk_models/model_en/ivector/final.ie
LOG (VoskAPI:ComputeDerivedUars():ivector-extractor.cc:183) Computing derived variables for iVector extractor
LOG (VoskAPI:ComputeDerivedUars():ivector-extractor.cc:204) Done.
LOG (VoskAPI:ReadDataFiles():model.cc:282) Loading HCL and G from /home/root/vosk_models/model_en/graph/HCLr.fst /home/root/vosk_models/model_en/graph/Gr.fst
LOG (VoskAPI:ReadDataFiles():model.cc:308) Loading winfo /home/root/vosk_models/model_en/graph/phones/word_boundary.int
LOG (l_869.422057) imx_sph0645_startup():
VoskAPI:UpdateGrammarFst():recogni_869.4273021 imx_sph0645_trigger():
izer.cc:287) ["entrance", "on", "down", "exit"]
LOG (VoskAPI:Estimate():language_model.cc:142) Estimating language model with ngram-order=2, discount=0.5
LOG (VoskAPI:OutputFst():language_model.cc:209) Created language model with 5 states and 8 arcs.

PREU presionado
[ 873.746491]
[ 873.749557]
[ 875.147181]
[ 875.152777]
PREU liberado
Setting pipeline to PAUSED ...
Pipeline is live and does not need PREROLL ...
Pipeline is PREROLLED ...
Setting pipeline to PLAYING ...
New clock: GstSystemClock
[ 875.972863] ov5645_mipi_v2 1-003c: ov5645_enable_af: af_mode = 2
[ 876.231323] skip frame 1
[ 876.277982] skip frame 1
[ 876.364662] skip frame 1
Redistribute latency...
PLAY presionado99:99.
PLAY liberado
ACI presionado
PAIR presionado
OCI presionado
...

Reconocido: entrance
'on' para encender el LED o 'down' para apagarlo.
Reconocido: on
LED ON
Reconocido: down
LED OFF
Reconocido: on
LED ON
Reconocido: down
LED OFF
ACI presionado
Reconocido: down
LED OFF
Reconocido: exit
Saliedo del asistente de voz.
Reconocido: on
Reconocido: down
```

Activación de VOSK y Botones

Botones

Comandos de voz

Problemáticas encontradas y su solución

Reconocimiento de voz en la board

Tras elaborar el código de reconocimiento de voz para la PC, encontramos diversos retos para adaptarlo a la board, entre ellos:

La diferencia de muestreo necesaria entre el micrófono de la board y el reconocido por VOSK, por lo cual fue necesario realizar lo siguiente:

Definir las frecuencias a las que trabaja nuestro hardware y el modelo de VOSK.

```
rate_hw = 32000          # Frecuencia de SPH0645
rate_vosk = 16000        # Frecuencia de Vosk
```

Se define como va a funcionar nuestro micrófono en la tarjeta, es decir, se aplica el comando arecord para grabar, se define el HW del micrófono, su formato de grabación, la frecuencia de grabación, el modo de grabar de un canal y que no se mande a un archivo aparte.

```
# --- Ajustes para microfono ---
cmd = [
    "arecord",
    "-D", "hw:0,0",          # SPH0645
    "-f", "S32_LE",
    "-r", str(rate_hw),      # 32 kHz
    "-c", "1",               # Mono
    "-q",                    # Silencioso
    "-t", "raw"
]
```

Se normaliza el audio para que el asistente de voz escuche bien en diversas condiciones.

```
# --- Normalizar audio ---
def normalize_audio(data_32):
    data_f = data_32.astype(np.float32)
    rms = np.sqrt(np.mean(data_f ** 2))
    if rms < 1:
        rms = 1
    gain = 20000.0 / rms
    data_norm = np.clip(data_f * gain, -32768, 32767)
    return data_norm.astype(np.int16)
```

Se aplican los procesos de normalización y conversión de formatos de manera constante para que el asistente de voz se mantenga detectando adecuadamente los comandos.

```
# Conversion y muestreo
data_32 = np.frombuffer(data, dtype=np.int32)
data_16 = normalize_audio(data_32)
data_16_resampled = data_16[::2] # 32k ? 16k
```


Otro problema encontrado era la confusión de palabras detectadas en el i.MX, por lo tanto se realizó lo siguiente:

```
recognizer = KaldiRecognizer(model, rate_vosk, ["entrance", "on", "down", "exit"])
```

Con este arreglo fue posible limitar la cantidad de palabras que entiende el modelo, para que al decir la WakeWord y comandos no los confunda con otras palabras.

Finalmente para corregir la acción de cerrar el vocie asientant con “Exit”: se cambio la variable `is_active` por `false`, en lugar de aplicar un `break`, esto hace que no se cierre el asistente de voz, si no que solamente salga del bucle que aplica los comandos, esperando nuevamente la mención de “Entrance” Para que `is_active` sea `true` y se puedan aplicar los comandos de voz.

```
elif "exit" in text:          # Cerrar asistente de voz hasta nuevo entrance
    print("Saliendo del asistente de voz.")
    is_active = False
```

Dando como resultado un asistente de voz funcional.

```
LOG [16:39:12.727] imx_sph0645_trigger():
VoskAPI:UpdateGrammarFst()>recognit 3258.729113] imx_sph0645_trigger():
izer.cc:287] ["entrance", "on", "down", "exit"]
LOG <VoskAPI:Estimate()>:language_model.cc:142> Estimating language model with ngram-order=2, discount=0.5
LOG <VoskAPI:OutputToFst()>:language_model.cc:209> Created language model with 5 states and 8 arcs.
Escuchando microfono <SPH0645>... di 'entrance' para activar control de voz
Reconocido: entrance
Voice control activado. Di 'on' para encender el LED o 'down' para apagarlo.
Reconocido: on
LED ON
Reconocido: on
LED ON
Reconocido: down
LED OFF
Reconocido: down
LED OFF
Reconocido: exit
Saliendo del asistente de voz.
Reconocido: on
Reconocido: on
Reconocido: down
Reconocido: down
Reconocido: entrance
Voice control activado. Di 'on' para encender el LED o 'down' para apagarlo.
Reconocido: on
LED ON
Reconocido: down
LED OFF
Reconocido: exit
Saliendo del asistente de voz.
```

Estado actual del proyecto

Etapas completas:

- Detección de botones.
- Detección de led externo.
- Archivo con audio de timbre (timbre.wav)
- Funcionamiento de la cámara.
- Reproducción de audio.
- Detección de wakeword en Linux (máquina virtual).
- Detección de comandos de voz en la board.
- Botones + cámara.
- Integración de funcionamiento de HW + Reconocimiento de voz.

Dentro del código buttons.c, se da la instrucción de que al presionar el botón PREV, se reproduzca el sonido del timbre, mediante la ejecución del script doorbell.sh, el cual contiene el binario de playback, además, activa la cámara, mediante camera.sh, por lo tanto simula la acción esperada de que cuando una persona timbre a la casa, suene una alerta y también abra la cámara para identificar quien es.

Con el botón PLAY se cierra la cámara, permitiéndole al usuario el tiempo suficiente para identificar a la persona que esta fuera y con ello el mismo decidir cuándo apagar la cámara, mediante un comando pkill que termina el proceso de gst.

Con el botón ACT se enciende el led D5 del voicehat mediante la ejecución del script led_on.sh, simulando el encender la luz de la casa de manera manual, mientras que con PAIR se ejecuta led_off.sh , lo cual apaga el led, simulando el apagar la luz de la casa del usuario.

Con esto se puede concluir que se tiene un buen avance respecto a la integración del funcionamiento del proyecto, especialmente en la parte del hardware.

Mientras tanto en el área de voice recognition, es posible apreciar que ya hay un reconocimiento claro y preciso dentro de la board i.MX8M Mini. Se resolvieron problemas técnicos como la diferencia de frecuencias de muestreo entre el micrófono y el modelo VOSK, la confusión de palabras detectadas dentro del modelo y la integración del prendido y apagado del led mediante el reconocimiento de voz, obteniendo la versión funcional del asistente de voz necesario para el proyecto.

Bibliografía:

- [1] *Build a voice assistant with TensorFlow Lite*. (n.d.). MOUSER ELECTRONICS. https://www.mouser.mx/applications/voice-assistant-tensorflow-lite/?srsltid=AfmBOoqIgOOojkEjI4qsckqGJbfltrK1Vv7AKROWzbGb9Xa6ZVngAln&_gl=1*27pggi*_ga*MTg4NzIyOTU4LjE3NTczMTMxNDY.
- [2] <https://www.openvoiceos.org/>. (n.d.). <https://www.openvoiceos.org/>.
- [3] Nxp-Imx-Support. (n.d.-b). *GitHub - nxp-imx-support/imx-voiceplayer: This is an application for controlling an audio player using the Bluetooth communication protocol by the use of voice commands. It is intended to be included in NXP demo experience*. GitHub. <https://github.com/nxp-imx-support/imx-voiceplayer>
- [4] NetworkChuck. (2024, November 4). *my local, AI Voice Assistant (I replaced Alexa!!)* [Video]. YouTube. <https://www.youtube.com/watch?v=XvbVePuP7NY>
- [5] Fernández, Y. (2025, 14 julio). *Qué es Alexa, qué puedes hacer con él y qué dispositivos son compatibles*. Xataka. <https://www.xataka.com/basics/que-alexa-que-puedes-hacer-que-dispositivos-compatibles>